

1 Using the Repository as Development Tool

If you use the MDS Repository as development tool to create new services, you must mind the information given in the following to correctly use the tool.

1.1 Use of transformation types for the dynamic conversion of DB or BAPI values

1.1.1 Overview

With interpreted services, you use transformation types to transform service parameters for the integration in database tables or for the integration as PDM dialog parameters.

Example: A Boolean service parameter can be mapped in the database as 1/0 and/or as J/N as parameter of the PDM dialog.

You can use transformation types for interpreted wrappers (service calling a PDM dialog) and for interpreted list services (service directly accessing the database).

The runtime interpreters integrate particular "special treatments" in their standard form (e.g. converting the string fields returned by the PDM into the data type according to repository). Other things cannot be generally integrated because they do not always work the same way (e.g. conversion of Boolean values. These values are sometimes mapped by J/N or by 1/0 in the PDM dialog or database).

The service parameters in the repository provide a column named Transformation Type. Enter the definition of the transformation in a key/value format in this column.

Example:

```
FCT=BOOLTRANSFORMATION | TRUEINVAL=N | FALSEINVAL=J | TRUERESVAL=N | FALSERESVAL=J |
```

You must always specify the value **FCT=...** that assigns the function. Depending on the function, other values must additionally be specified to configure the function.

1.1.2 Standard transformation functions

HYCOLORTORGBTRANSFORMATION

With wrappers and list services, you use this transformation to convert fields, which contain a color in PDM color code (1-16), for the return into the relevant RGB presentation as integer.

FCT-Id

HYCOLORTORGBTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Conversion of PDM color codes from the string field into RGB as integer
List Result	Selection of the PDM color code as integer from the database and then conversion into RGB as integer

DATETIMEFILTERTRANSFORMATION

This transformation is used with list services to deposit a filter for fields, which are selected as datetime via the database function get_datetime, but which consist of two separate fields including the date and the time component in the database.

FCT-Id

DATETIMEFILTERTRANSFORMATION

Configuration parameters

Name	Description
DBFIELDDATE	Name of database field with date component (without alias)
DBFIELDTIME	Name of database field with time component (without alias)

Supported transformations

Transformation	Description
List Call	Adding a SeparateDateAndTimeFilter for the field, configured with the two specified database fields

BOOLTRANSFORMATION

You use this transformation to process Boolean fields for wrappers and list services. The processing integrates the implementation for the PDM call, the list service filter and the result conversion with both service types.

FCT-Id

BOOLTRANSFORMATION

Configuration parameters

Name	Description
TRUEINVAL	(OPTIONAL) Value for Yes (Ja) when calling PDM dialog/list filter (default J or if REALDATATYPE=integer then 1)
FALSEINVAL	(OPTIONAL) Value for No when calling PDM dialog/list filter (default N or if REALDATATYPE=integer then 0)
TRUERESVAL	(OPTIONAL) Value for Yes (Ja) when returning a PDM dialog/the selection from the database (default J or if REALDATATYPE=integer then 1)
FALSERESVAL	(OPTIONAL) Value for No when returning a PDM dialog/the selection from the database (default N or if REALDATATYPE=integer then 0). Must include a value matching the specified REALDATATYPE (J for REALDATATYPE=integer is an error)
REALDATATYPE	(OPTIONAL) Specifies the real data type of the field; integer and string are supported (default string)
NULLHANDLING	(OPTIONAL) Specifies the interpretation of null values. Possible values are none (ignore null), true (interpret null as true) and false (interpret null as false) (default none)
OTHERVALHANDLING	(OPTIONAL) Specifies the interpretation of other values (than null, the value for true and the value for false). Possible values are none (ignore others), true (interpret others as true) and false (interpret others as false) (default none)

Supported Transformations

Transformation	Description
PDM Result	Converts the string value provided by the PDM Result into Bool
List Result	Converts the selected value from the DB (integer or string) into Bool
PDM Call	Converts the true/false from the client call into the configured true/false values
List Call	Adds a filter for the DB field to filter the SQL according to data type and configured true/false values.

Examples:

Real value of string type, true=Y and false=N; null is interpreted as null (NULLHANDLING, REALDATATYPE, FALSEINVAL and FALSERESVAL need not be specified because default values are correct)

```
FCT=BOOLTRANSFORMATION | TRUEINVAL=Y | TRUERESVAL=Y |
```

Real value of integer type, true=1 and false=0; null is interpreted as null (NULLHANDLING, TRUEINVAL, TRUERESVAL, FALSEINVAL and FALSERESVAL need not be specified because default values are correct)

```
FCT=BOOLTRANSFORMATION|REALDATATYPE=integer|
```

DECIMALPLACESTRANSFORMATION

Converts the decimal places of a wrapper, e.g. "2" into a FormatString, in this case "#####0.##"

FCT-Id

DECIMALPLACESTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
PDM Call	Conversion of a number into a FormatString, "2" -> "#####0.##".

DECIMALPLACESNUMBERTRANSFORMATION

With a list service, converts a FormatString, e.g. "0.00" into an integer, in this case "2".

FCT-Id

DECIMALPLACESNUMBERTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
----------------	-------------

List Result	Converts a FormatString, e.g. "0.00" into an integer, "0.00" -> "2".
-------------	--

CASEINSENSITIVEFILTERTRANSFORMATION

String filter on a DB field regardless of upper/lower case

FCT-Id

CASEINSENSITIVEFILTERTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
List Call	String Filter Case insensitive

SHIFTENDFILTERTRANSFORMATION

This transformation is used to filter by the time stamp of shift end in list services. Here, the date field or the date field + 1 day must be used, depending on whether the shift end is larger or smaller than the shift start.

FCT-Id

SHIFTENDFILTERTRANSFORMATION

Configuration parameters

Name	Description
DBFIELDDATE	Name of database field with shift date (without alias)
DBFIELDBEGIN	Name of database field with shift start (without alias)
DBFIELDEND	Name of database field with shift end (without alias)

Supported Transformations

Transformation	Description
List Call	Adds a ShiftEndDateFilter using shift date, start and end

DATATYPECONVERSIONTRANSFORMATION

This transformation is used to convert the data type between DB and web service with list services.

At present, the following are supported:

- string to integer
- string to decimal
- integer to string
- integer to decimal
- decimal to string
- decimal to integer

FCT-Id

DATATYPECONVERSIONTRANSFORMATION

Configuration parameters

Name	Description
DBTYPE	Data type in DB
WSTYPE	Data type for web service

Supported Transformations

Transformation	Description
List Call	If filters are specified in web service type, the filters are converted into DB type.
List Result	Conversion of DB type field into web service type in result.

TSPARTTRANSFORMATION

This transformation is used to identify the components of a time stamp in list services. Components are, e.g. year, day, months, calendar week,

FCT-Id

TSPARTTRANSFORMATION

Configuration parameters

Name	Description
MODE	Component to be identified (see separate table)

Supported Transformations

Transformation	Description
List Result	Conversion of DB type field datetime into the required component (as integer) in the result.

Valid Values for MODE

Name	Description
DAY	Day of month
DOY	Day of year
DOW	Day of week (0=Sunday ... 6=Saturday)
MONTH	Month
YEAR	Year
CWJ	Calendar week acc. to JAVA standard (CW1 = first complete week in year)
CWD	Calendar week acc. to DIN 1355/ISO 8601 (CW1 = week including January 4th)
CWU	Calendar week acc. to USA (CW1 = week including January 1st)
QUART	Quarter
HR	Hour
MIN	Minute
SEC	Second
MIL	Millisecond
MONTHB	Month of business year. The first month of the business year is identified via CAQ Option 1018.
QUARTB	Quarter of business year. The first month of the business year is identified via CAQ Option 1018.

Examples:

Domain	Service Function	Acronym	Is Result	Web Service Type	Conversion Method	Data Object Name	DB Alias	DB Field
Click here to add a new row								
TsPartTest	list	a.ts	☒	datetime		TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.day	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DAY	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.doy	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DOY	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.month	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MONTH	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.year	☒	integer	FCT=TSPARTTRANSFORMATION MODE=YEAR	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.cwj	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWJ	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.hr	☒	integer	FCT=TSPARTTRANSFORMATION MODE=HR	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.min	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MIN	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.sec	☒	integer	FCT=TSPARTTRANSFORMATION MODE=SEC	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.mil	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MIL	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.dow	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DOW	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.quart	☒	integer	FCT=TSPARTTRANSFORMATION MODE=QUART	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.cwd	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWD	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.cwu	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWU	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.monthb	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MONTHB	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)
TsPartTest	list	a.quartb	☒	integer	FCT=TSPARTTRANSFORMATION MODE=QUARTB	TsPartTest.list	ut	hydadm.get_datetime(%1\$.bearb_date,%1\$.bearb_time)

ALPHAPERSONIDTRANSFORMATION

This transformation is used with list services to convert an alphanumerical personnel number into a numerical number for the result, and/or to filter the alphanumerical field using the numeric number.

FCT-Id

ALPHAPERSONIDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
List Call	Adds a special filter in order to filter the alphanumerical field using the numeric person.id.
List Result	Conversion of DB type string field into numeric personnel number

FIXCUTOFFNUMWORKPLACEIDTRANSFORMATION

This transformation is used to fill the result field with leading zeros. This is required with systems using numeric machine numbers and wrappers that have cut off the leading zeros. But if the client requires the complete machine number like it is included in the database, this transformation is used.

FCT-Id

FIXCUTOFFNUMWORKPLACEIDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Completes the relevant result field with leading zeros until 8 characters are reached, if the result field is shorter and numeric machine numbers are active.

CATEGORYLEDTRANSFORMATION

With wrappers and list services, this transformation is used to convert fields, which contain the name of an order category bitmap, for the return into the LED constant.

The assignment is as follows:

Bitmap	LED constant
fa.bmp	LED_FA
gk.bmp	LED_GK
kp.bmp	LED_KP
na.bmp	LED_NA
pj.bmp	LED_PJ
pm.bmp	LED_PM

FCT-Id

CATEGORYLEDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Conversion of the bitmap name from the string field of the PDM result into the LED constant
List Result	Selection of the bitmap name as string from the database and subsequent conversion into LED constant

STATUSLEDTRANSFORMATION

With wrappers and list services, this transformation is used to convert fields, which contain the name of a status bitmap, for the return into the LED constant.

The assignment is as follows:

Bitmap	LED constant
x.bmp	LED_RED
v.bmp	LED_GREY
u.bmp	LED_YELLOW
p.bmp	LED_BLUE
n.bmp	LED_PINK
l.bmp	LED_LIGHT_GREEN
f.bmp	LED_YELLOW_GREEN
e.bmp	LED_GREEN
a.bmp	LED_BLACK

FCT-Id

STATUSLEDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Result	Conversion of the bitmap name from the string field of the PDM result into the LED constant
List Result	Selection of the bitmap name as string from the database and subsequent conversion into LED constant

LEGACYFULLTSTRANSFORMATION

With wrapper services, this transformation is used to map a complete time stamp to a single acronym of the dialog string. Using the default functions of the wrapper interpreter, you can only map the date component to an acronym or date and time each to separate acronyms.

The values of the time stamp are assigned in the format MM/dd/yyyy HH:mm:ss to the acronym.

FCT-Id

LEGACYFULLTSTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
PDM Call	Setting the complete time stamp to an acronym

LEGACYARRAYPARAMETERTRANSFORMATION

This transformation is used to support an "IN" and "BETWEEN" with wrapper services. Using the default functions of the wrapper interpreter, you can only map single values to PDM acronyms.

The list of values is converted into a string separated by separators. Each single value is also embraced by single inverted commas for the "string" web service type. The separator used between the single values can be configured. By default, it is a comma.

FCT-Id

LEGACYARRAYPARAMETERTRANSFORMATION

Configuration parameters

Name	Description
SEPARATOR	Optional: separator used, if not specified, then comma

Supported transformations

Transformation	Description
PDM Call	Conversion of value lists into a string separated by separators



Only the data types "string" and "integer" are supported!

DATEONLYFILTERTRANSFORMATION

With list services, you use this transformation to only use the date component as filter and to ignore a time component that might exist. This way, you can document in the interface that only a date component is processed and a client need not remove the time component itself.

FCT-Id

DATEONLYFILTERTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported transformations

Transformation	Description
List Call	Adds a filter that only uses the date component and ignores the time component.

1.2 Checklist: Repository data

The correct completion of the MDS repository is a complex task, which is occasionally prone to errors, too. The following sections might help you to avoid typical mistakes.

Term definition: *Input parameter* is an acronym, if **isFilterParameter** or **isSpecialParameter** is set. *Output parameter* or *Return value* is an acronym, if **isResult** is set to Y.

ServiceParameter: An input parameter must define at least one operator

Effect: Wrapper generator stops with the following error message: Input Parameter specified with no operator (with specification of relevant acronym and services)

Reason: An input parameter must define at least one operator.

Solution: Set at least one of the Can columns (usually CanEqual) to Y

ServiceParameter: Operator cannot exist without input parameters

Effect: Wrapper generator stops with the following error message: Operator specified for parameter that is no input parameter (with specification of relevant acronym and services)

Reason: An input parameter must define at least one operator.

Solution: Set at least one Can column (usually CanEqual) to Y.

ServiceParameter: Acronym must be input or output parameter

Description: it is not possible that an acronym is neither input nor output parameter, exception: acronym with fixed value for the wrapper.

Effect: Wrapper generator stops with the following error message: Neither input nor output parameter found (with specification of relevant acronym)

ServiceParameter: Acronym with fixed value for the wrapper

Description: Normally, an acronym is at least one of both: input or output parameter. If a wrapper must transfer a parameter with fixed value after a BAPI call and if the client does not know this, then all three columns for input and output parameters may remain unset.

Check: isFilterParameter, isSpecialParameter, isResult blank, DefaultValue must be set; HydraAcronym must be set.

Effect: if DefaultValue is not set, the wrapper generator stops with the following error message: Neither input nor output parameter found (with specification of relevant acronym)

For details, refer to section 0.

ServiceParameter: Acronym with fixed value for wrapper must have string data type

Description: WebServiceType for acronym with fixed value may only be string, even if DB column has a different data type.

Effect: Wrapper generator stops with the following message: java.lang.IllegalStateException: Fix value parameter with other datatype than string found: INTEGER

Solution: Declare data type in repository as string.

ServiceParameter: Reference field for *date, *time and datetime must be identical

Effect: Wrapper generator stops with the following error message: At least one component of date/time triple parameter is missing

Reason: The entries with the * types are specified for the wrapper service and must have the same reference value as the corresponding datetime entry. The reference value must be unique within a service.

Solution: set the three reference values to an equal value.

ServiceParameter: Hydra acronym must be available for an input parameter of a wrapper service

Effect: Wrapper generator stops with the following error message: Input parameter with empty Hydra-Acronym found.

Reason: The wrapper service must be able to map an acronym to the HYDRA acronym of the BAPI

Solution: Add HYDRA acronym

ServiceParameter: A wrapper service must have at least one parameter

Effect: an error is only produced if you call update, delete, lock, or unlock in the client application

Reason: Functions as delete or lock usually require a key field

ServiceParameter: Wrapper services do not have any output parameters

Effect: Description: isResult must not be set

Solution: empty isResult

ServiceParameter: WrapperServices do not have any filter parameters

Description/Solution: Set isSpecialParameter to Y, empty FilterParameter

ServiceParameter: No double acronyms within the same service

Exception: multiple ResultSets

Effect: DataLogic generator stops with the following error message: ERROR: Acronym duplicate in non-multiple result set: <acronym>! Please ensure the services.xml export doesn't contain duplicate entries!

Solution: Check each service for clear acronyms

ServiceParameter: Specify key field also for list service

Effect (e.g.): If you use Delete in the MOC application, this causes the exception "MissingPrimaryKeyException". But isKey is available in a correct form in repository, wrapper, servicex.xml and DataLogic of the delete service. The reason is that a key is missing in the list service so that the data record can be identified in the grid.

Solution: Specify isKey and isMandatory for the key fields of the list

ServiceParameter: Specify mandatory fields for wrapper

Effect: Insert service of client application complains and shows an error returned from BAPI.

Solution: Set isMandatory for the respective mandatory fields (see relevant SystemDesign document)

Service: Set service type correctly

Specific case: all services of the domain (list, insert, copy, update, delete, lock, ...) are set to JavaService although wrappers are planned.

Effect: ProjectManager/SVN Working Copy/Commit does not suggest the newly "created" WrapperServices for check-in.

Reason: The exported services.xml is empty when it is created; WrapperGenerator runs without error message, but does not generate any source code, which is completely correct.

Solution: visual diagnosis. List is often a JavaService, but the other functions of the domain are wrappers.

All tables, all columns with specific value stock: only particular specified values are permitted

Typical: V instead of Y. This is very difficult to identify in a visual check.

This can happen with Insert/Paste (Ctrl-V) into the repository.

Solution: Check columns for not permitted values (in case of visual diagnosis, use column filter selection).

1.3 InterpretedWrapper: Transfer of fixed values to PDM dialog

If fixed values must be transferred (e.g. MOD=E) with a service of type InterpretedWrapper to the PDM dialog (independent of client call), then the values must be entered as follows:

- WebServiceType: set to string.
- DefaultValue: (here the default value must be entered, E for example)
- HydraAcronym: (here the acronym must be entered, MOD for example)

The following columns MUST be empty:

- InputAsArray
- IsSpecialParameter
- IsFilterParameter
- IsMandatory
- Can....

- IsResult